Chiron

Early Orchestration Direction and Initial System Realization

Progress Report 3 • 27 Oct 2025 — 9 Nov 2025

Master’s Research Progress Update

Fahad Alothman

Progress Report 3

Title: Early Orchestration Direction and Initial System Realization
Reporting Window: 27 Oct 2025 — 9 Nov 2025
Submission Context: Master’s Research Progress Update

Abstract

This third progress report closes the initial reporting phase by documenting the transition from research-backed design foundation into an initial implementation and orchestration direction. The previous two reports established the central problem—supporting better software development lifecycle decisions under agentic workflows—and clarified that the solution required structured guidance, visible progression, and stronger design and domain foundations. This report captures the next step: beginning to materialize those ideas into a working system direction.

The main outcome of this period was not yet a fully realized guidance platform, but a concrete shift from conceptual architecture into initial implementation surfaces. The project began transforming design artifacts and methodological assumptions into code-level commitments: schema implementation, seeded agent roles, initial user-facing application structure, and an early orchestration stack that could support guided workflow execution. The report also records why this shift mattered: once a system begins to materialize, assumptions that felt stable in research become testable and sometimes fragile.

---

1. Why Progress Report 3 Matters

Progress Report 3 marks the point where the project ceased to be only a comparative and design exercise. In the earlier reports, the main question was how to frame the problem of guidance for AI-assisted software development. In this period, the question became more concrete:

> What is the minimum working system direction that can begin to embody that guidance thesis without prematurely overcommitting to a final architecture?

This distinction is important. The project was still in an exploratory phase, but exploration was no longer only conceptual. It had become implementation-oriented exploration: enough to test whether the emerging design and methodology assumptions could survive contact with real system structure.

2. From Design Foundation to Build Direction

In this period, the project moved from design-backed planning into initial system realization. Several themes converged:

- the domain model was no longer just an abstraction and began to serve as a practical implementation boundary,
- workflow execution was no longer only a conceptual future requirement and began to influence concrete technical choices,
- and the project began defining what a guided development environment would need in terms of execution surfaces, role definitions, and orchestration patterns.

This was not yet the stage where later formal guidance layers had crystallized. Instead, the period should be understood as the first serious attempt to turn a research thesis into a buildable system direction.

---

3. Initial System Commitments

3.1 Schema and Persistent Structure

One major shift in this period was that domain modeling began to translate into implementation commitments. The project moved toward a concrete 16-table schema that represented projects, workflows, workflow paths, execution records, agent definitions, approvals, outputs, and variable context.

This matters because it shows a change in the mental model of the project:

- from “the system should help people follow a process”
- to “the system must persist the entities that make that process inspectable and recoverable.”

3.2 Initial User-Facing Application Direction

At this stage, the application direction increasingly reflected a guided workbench model. Rather than treating the user as someone who only chats with an agent, the project began emphasizing:

- project-level navigation,
- workflow-specific views,
- execution visibility,
- and workbench-like artifact interaction.

This was still early, but it is a key threshold: guidance was beginning to be translated into interface commitments, not just methodological slogans.

3.3 Role and Workflow Execution Thinking

The earlier BMAD baseline had already reinforced role-aware task decomposition. In this report period, that idea became more operational: the project began treating workflows and agent roles as parts of an executable system, rather than as planning-only aids. Early seeding direction for core roles such as PM, Analyst, Architect, Developer, Scrum Master, and UX-oriented roles reflects that shift from conceptual process participants into concrete system entities.

This is the first point where “agent orchestration” becomes a concrete architectural direction rather than a purely aspirational phrase.

3.4 Methodology Starts Becoming Data-Driven

Another notable change in this period was that methodology began to move away from hardcoded assumptions and toward data-backed execution support. Dynamic options, structured tags, and workflow metadata started to matter more as operational inputs rather than as loose supplementary notes. This was not yet the later formal methodology architecture, but it was the first clear indication that if methodology was meant to guide execution, it would eventually need to exist as system data rather than only explanatory process material.

---

4. Why Orchestration Became Central

By this period, the project had enough evidence to conclude that the guidance problem could not be solved purely by good specs, good prompts, or better chat ergonomics. Those were helpful, but insufficient. The real challenge was how to coordinate:

- multiple stages of work,
- multiple role responsibilities,
- evolving project context,
- and user oversight over progress.

This made orchestration central. However, the orchestration goal at this stage was still modest and early:

- define a path for workflows to execute in a guided sequence,
- preserve enough context to move across steps coherently,
- and support the user in following the project lifecycle without reconstructing everything manually.

The project had not yet reached its later more formal methodology framing. What it did achieve here was a decisive realization that orchestration itself was part of the thesis contribution.

---

5. Tool and Framework Perspective in This Period

This reporting window is also the first point where the project's relation to the agentic tooling landscape became more concrete.

5.1 BMAD as Baseline, Still Valuable

BMAD remained highly relevant in this period because it provided a structured way to move from ideation into implementation planning. It was still the strongest baseline under consideration because it offered not just specs, but specifications coupled with workflows, role specialization, and guided process movement.

5.2 Chat-Mediated Guidance Still a Friction Point

At the same time, the limitation identified in Progress Report 2 remained active: guidance was still too dependent on repeated chat interactions. Even when the process was structured, users often had to repeatedly ask the system what had been completed, what remained next, and how current work related to the broader project state. This meant that the system still relied too much on conversational recovery of progress, rather than making progress directly visible.

5.3 Comparison Context: Orchestration and Planning Tools

This period also benefited from a broader awareness of other approaches in the ecosystem:

- **CrewAI** represented a strong multi-agent orchestration approach, emphasizing agents, tasks, crews, flows, planning, reasoning, memory, and collaboration [4], [5].
- **Vibe Kanban** represented a more UI-centered planning/review paradigm, especially around planning, prompting, reviewing, and running coding agents in parallel workspaces [6], [7].
- **Conductor** represented a more directly developer-facing model of parallel coding-agent orchestration with isolated workspaces, git-worktree-backed execution, and human review over resulting diffs [17], [18].

These were useful comparison points because they reinforced two distinct aspects of the problem:

- orchestration frameworks help coordinate autonomous or semi-autonomous agents,
- planning/review tools help externalize work and reduce human bottlenecks.

The project's early direction sat between these poles: it was concerned with both SDLC guidance and agent orchestration, rather than only one of them.

That means this report should not be read as if Chiron had already decided against isolated-workspace or sandbox-oriented execution ideas. At this stage, those ideas were still genuinely attractive and relevant. The more accurate historical point is that Chiron was exploring how to combine that orchestration paradigm with something stronger on the guidance side: a system that would not merely coordinate agents, but would gradually offload the project's mental model into structures shared between the user and the implementing agents.

---

6. Early Technical Direction and Initial Stack Thinking

By this point, the project was beginning to settle on a first serious technical direction for implementation. This included:

- a desktop-oriented user experience rather than a purely CLI-bound one,
- web technologies suitable for rich project/workflow interfaces,
- a backend and persistence layer capable of supporting explicit workflow and project entities,
- and an early orchestration approach that could support agentic execution in a structured way.

This period can therefore be read as the first point where implementation choices stopped being mere experiments and started becoming a coherent system direction.

At the same time, platform priorities were still flexible. The focus was on establishing the fastest workable implementation path that could test the orchestration thesis, rather than prematurely freezing the entire final delivery shape.

Importantly, however, these choices were still provisional enough to be revised later. The project was not yet declaring a final mature architecture; it was identifying an initial architecture that could carry the research problem forward into real testing.

---

7. Assumptions Strengthened or Revised in This Period

Assumption A

Assumption carried from Progress Report 2: Guidance must be visible, not merely implied.
Status in this period: Strengthened.
Why: Initial application and schema direction reinforced that visibility requires persistent system surfaces.

Assumption B

Assumption: Domain modeling and workflow planning are enough to preserve project continuity.
What challenged it: Early implementation thinking suggested that execution coordination itself must become a first-class concern.
Correction: Guidance systems need orchestration, not only planning structure.

Assumption C

Assumption: A strong methodology baseline can be translated directly into a product with minimal reinterpretation.
What challenged it: Translating structured methodology into user-facing software exposed interface and execution questions that methodology alone did not fully answer.
Correction: Methodology must be operationalized through system-specific design and execution boundaries.

Assumption D

Assumption: Research-stage architecture can stay abstract until later.
What challenged it: Once the system began to materialize, abstraction alone was no longer enough to decide what had to be persisted, executed, or surfaced.
Correction: The project needed an initial working architecture direction, even if later reports would refine or revise it.

---

8. Timeline (Progress Report 3 Scope)

Week 1 (27 Oct — 2 Nov 2025): Transition from Planning to Initial Build Direction

- Moved from design and planning artifacts into initial implementation commitment.
- Began treating workflows, agents, and execution context as persistent system concerns rather than only planning abstractions.
- Strengthened the idea of a guided workbench-like system rather than a purely chat-based assistant interface.
- Began implementing the 16-table schema and concrete seeding direction for core project roles.

Week 2 (3 Nov — 9 Nov 2025): Early Orchestration and Application Structure

- Advanced schema and application structure toward a first workable foundation.
- Clarified that the project required orchestration support, not just better planning artifacts.

- Crossed an implementation-readiness threshold: enough clarity existed to move from design-backed planning into disciplined build work.
- Ended the period with a clearer early product/system direction that could support future implementation work.

---

9. What This Period Still Had Not Become

It is equally important to state what had **not** yet materialized in this reporting window.

This period did **not** yet include:

- the later L1/L2/L3 methodology layering,
- a formal state-machine model,
- methodology versioning or progressive seeding,
- deterministic publish or project pinning semantics,
- or the later hardened facts/artifacts methodology structure.

What existed instead was a much earlier stage of the same broader trajectory: workflow execution thinking, role-aware orchestration, schema-backed persistence, and the first movement toward data-driven methodology support. This distinction keeps the report historically honest while still showing the project's direction of travel.

---

10. Why This Period Matters in the Larger Story

Progress Report 3 is where the project first becomes concretely legible as a system initiative rather than only a research thesis. The importance of this period is that it translates prior research into initial build commitments without pretending that the final form was already known.

This is also the point where SDLC guidance and agent orchestration become explicitly linked: guidance requires more than documentation, but orchestration requires more than raw autonomy. The project's emerging contribution lies in trying to combine those two concerns into one system direction.

---

11. Initial Stack Decision

To make the timeline explicit, this report now closes the initial reporting phase with the initially chosen implementation stack and the reasons for that choice at that time.

11.1 Initial Stack

Frontend / Desktop shell - React-based frontend workbench - Tauri desktop host (including sidecar discussions for local runtime/service coupling) [8]

Backend / orchestration + contracts - Python backend direction - FastAPI service boundary [9] - Pydantic for typed request/response and validation contracts [10] - DSPy-oriented orchestration/optimization exploration [1]–[3]

11.2 Why This Was Chosen in the Initial Phase

This decision optimized for research velocity and controllable local experimentation:

- **Python ecosystem speed for AI prototyping:** FastAPI + Pydantic provided rapid, strongly-typed API iteration [9], [10].
- **Local-first desktop intent:** Tauri gave a lightweight desktop host while preserving a web-style UI workflow [8].
- **Agentic experimentation fit:** DSPy-style experimentation aligned with the thesis goal of structured, improvable agent behaviors [1]–[3].
- **Pragmatic separation of concerns:** desktop shell and backend orchestration could evolve somewhat independently while the methodology model was still forming.

This was an intentionally provisional stack decision: good for proving feasibility and shaping the product thesis, but not yet guaranteed to be the long-term production architecture.

11.3 Transition Pressure Identified at End of the Initial Phase

By the end of the initial phase, one practical implementation pressure was already visible: development agents performed better in a single-language project than in a mixed-language runtime split. In practice, coordinating frontend/runtime code in TypeScript while maintaining a separate Python service path increased implementation friction for agent-driven development workflows.

This did not invalidate the initial stack decision, but it did create a clear signal for the next phase: moving toward a more unified TypeScript path would likely improve implementation throughput, consistency, and agent reliability.

11.4 Early Optimizer Research Direction

This period also seeded a line of optimizer research that would matter more clearly later when the AX and DSPy-style direction became more concrete. At the time, the project was not yet committing to one optimizer as implementation policy, but it was already useful to distinguish between three different optimization patterns.

MiPRO appeared attractive for structured, evaluable outputs because it jointly optimizes instructions and few-shot demonstrations, using bootstrapping and Bayesian search to improve prompt-program behavior against a defined metric [11], [12]. In Chiron terms, this made it a plausible fit for earlier workflow-path selection and other constrained routing or classification decisions where correctness could be scored.

GEPA represented a different kind of possibility. Rather than optimizing a single instruction/demo configuration, it uses reflective prompt evolution over execution traces and Pareto-style candidate selection, which makes it more suitable when competing objectives must be balanced explicitly [13], [14]. In Chiron, this suggested possible future use for outputs where quality, brevity, and downstream usefulness would need to be traded off rather than reduced to one simple score.

ACE pointed in yet another direction. Instead of focusing mainly on one-shot prompt optimization, it treats context as an evolving playbook through a generator-reflector-curator loop, making it especially relevant to long-running agents that accumulate and refine reusable rules over time [15], [16]. For Chiron, this aligned more naturally with the future idea of agents learning user-preferred practices across repeated workflows and multiple projects than with the immediate implementation baseline.

At this stage, these optimizer ideas should be read as research direction rather than settled implementation policy. The significance of this period is that the project was already distinguishing between prompt optimization for structured outputs, richer multi-objective evolution, and longer-horizon agent learning as separate design possibilities rather than collapsing them into one generic “AI optimization” bucket.

---

12. Risks and Open Questions Entering Progress Report 4

Risks

- Introducing implementation commitments too quickly before their guidance implications are understood.
- Building workflow execution features without sufficient clarity about how users observe and control them.
- Letting orchestration complexity outpace the clarity of the system’s visible model.

Open Questions

- What is the right balance between flexible agent autonomy and guided execution structure?

- Which orchestration responsibilities belong in workflow logic versus user-facing system design?
- How should the system present project progression so that users no longer need to reconstruct it through repeated chat queries?

These questions define the transition into Progress Report 4.

---

13. Conclusion

Progress Report 3 closes the initial reporting phase by documenting the first concrete step from research-backed design into initial system realization. The report does not claim that the final project architecture was already complete or stable. Instead, it shows that the project had matured enough to begin building toward its thesis direction in a disciplined way.

The period established that the core problem—guidance for software development lifecycle decisions under agentic workflows—requires both structured methodology and orchestration-aware system design. This insight created the conditions for later reports, where implementation decisions could become more explicit and technically grounded.

References

[1] O. Khattab et al., “DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines,” 2023. [Online]. Available: https://arxiv.org/abs/2310.03714
[2] K. Hu, “Learning AI Agent Programming (with DSPy),” Jun. 2025. [Online]. Available: https://blog.kevinhu.me/2025/06/22/Agentic-Programming/
[3] M. Mansurova, “Programming, Not Prompting: A Hands-On Guide to DSPy,” Towards Data Science, Jun. 23, 2025. [Online]. Available: https://towardsdatascience.com/programming-not-prompting-a-hands-on-guide-to-dspy/
[4] crewAIInc, “crewAI,” GitHub repository, 2025. [Online]. Available: https://github.com/crewAIInc/crewAI
[5] CrewAI, “CrewAI OSS: The open source, multi-agent orchestration framework,” 2025. [Online]. Available: https://crewai.com/open-source
[6] BloopAI, “vibe-kanban,” GitHub repository, 2025. [Online]. Available: https://github.com/BloopAI/vibe-kanban
[7] Vibe Kanban, “Orchestrate AI Coding Agents,” 2025. [Online]. Available: https://www.vibekanban.com/
[8] Tauri Programme, “Tauri,” Documentation, 2025. [Online]. Available: https://tauri.app/

[9] FastAPI, “FastAPI,” Documentation, 2025. [Online]. Available: https://fastapi.tiangolo.com/
[10] Pydantic, “Pydantic,” Documentation, 2025. [Online]. Available: https://docs.pydantic.dev/
[11] DSPy, “MIPROv2,” Documentation, 2025. [Online]. Available: https://dspy.ai/api/optimizers/MIPROv2/
[12] S. Opsahl-Ong et al., “Optimizing Instructions and Demonstrations for Multi-Stage Language Model Programs,” 2024. [Online]. Available: https://arxiv.org/abs/2406.11695
[13] DSPy, “GEPA Overview,” Documentation, 2025. [Online]. Available: https://dspy.ai/api/optimizers/GEPA/overview/
[14] L. A. Agrawal et al., “GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning,” 2025. [Online]. Available: https://arxiv.org/abs/2507.19457
[15] Q. Zhang et al., “Agentic Context Engineering: Evolving Contexts for Self-Improving Language Models,” 2025. [Online]. Available: https://arxiv.org/abs/2510.04618
[16] Ax LLM, “Agentic Context Engineering (ACE),” Documentation, 2025. [Online]. Available: https://axllm.dev/ace/
[17] Conductor, “Run a team of coding agents on your Mac,” 2025. [Online]. Available: https://conductor.build/
[18] Conductor, “Run a team of coding agents,” Documentation, 2025. [Online]. Available: https://docs.conductor.build/